PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013

Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

AI–Driven Public Health Chatbot For Disease Awareness

A PROJECT REPORT

Submitted by

K SUDEEP	-	20221CSE0591
TEJASWINI V BONGALE	-	20221CSE0625
SUVIDHA S IBERI	-	20221CSE0622

Under the guidance of,

Dr. Sreelatha P K

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

PRESIDENCY UNIVERSITY

BENGALURU

APRIL 2026

PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013
Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

Certified that this report "AI–Driven Public Health Chatbot For Disease Awareness " is a bonafide work of "K Sudeep (20221CSE0591), Tejaswini V Bongale (20221CSE0625), Suvidha S Iberi (20221CSE0622)", who have successfully carried out the project work and submitted the report for partial fulfilment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE AND ENGINEERING during 2025-26.

Dr. Sreelatha P K
Project Guide
PSCS
Presidency University

Mr. Muthuraju V
Program Project Coordinator
PSCS
Presidency University

Dr. Sampath A K
School Project Coordinator
PSCS
Presidency University

Dr. Blessed Prince P
Head of the Department
PSCS
Presidency University

Dr. N Duraipandian
Dean
PSCS
Presidency University

Examiners:

Sl. No.	Name	Signature
1.	Kalpana K Harish	24/4/26
2.	Dr. B. Chandrashekar	24/4/26

PRESIDENCY UNIVERSITY

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

We the students of final year B.Tech in COMPUTER SCIENCE AND ENGINEERING at Presidency University, Bengaluru, named K Sudeep, Tejaswini V Bongale, Suvidha S Iberi, hereby declare that the project work titled "AI–Driven Public Health Chatbot For Disease Awareness" has been independently carried out by us and submitted in partial fulfilment for the award of the degree of B.Tech in COMPUTER SCIENCE AND ENGINEERING during the academic year of 2025-26. Further, the matter embodied in the project has not been submitted previously by anybody for the award of any Degree or Diploma to any other institution.

K SUDEEP	20221CSE0591	
TEJASWINI V BONGALE	20221CSE0625	
SUVIDHA S IBERI	20221CSE0622	

PLACE: BENGALURU

DATE: 24th April 2026

ACKNOWLEDGEMENT

For completing this project work, we have received the support and the guidance from many people whom I would like to mention with deep sense of gratitude and indebtedness. We extend our gratitude to our beloved **Chancellor, Pro-Vice Chancellor, and Registrar** for their support and encouragement in completion of the project.

I would like to sincerely thank my internal guide **Dr. Sreelatha P K, Assistant Professor,** Presidency School of Computer Science and Engineering, Presidency University, for her moral support, motivation, timely guidance and encouragement provided to us during the period of our project work.

I am also thankful to **Dr. Blessed Prince P, Professor, Head of the Department, Presidency School of Computer Science and Engineering** Presidency University, for his mentorship and encouragement.

We express our cordial thanks to **Dr. N Duraipandian**, Dean PSCS & PSIS, Presidency School of computer Science and Engineering and the Management of Presidency University for providing the required facilities and intellectually stimulating environment that aided in the completion of my project work.

We are grateful to **Dr. Sampath A K, PSCS** Project Coordinators**, Mr. Muthuraju V,** Program Project Coordinator, Presidency School of Computer Science and Engineering, or facilitating problem statements, coordinating reviews, monitoring progress, and providing their valuable support and guidance.

We are also grateful to Teaching and Non-Teaching staff of Presidency School of Computer Science and Engineering and also staff from other departments who have extended their valuable help and cooperation.

K SUDEEP

TEJASWINI V BONGALE

SUVIDHA S IBERI

Abstract

The management of basic healthcare information in the modern world has become increasingly complex due to the rapid growth of digital technologies and the rising demand for accessible medical guidance. Many individuals face challenges in identifying symptoms, understanding possible diseases, and obtaining preventive advice, as healthcare information is often scattered across websites, applications, and unreliable sources. This lack of a unified and structured platform leads to confusion, misinformation, delayed decision-making, and increased dependency on physical consultations, especially in regions with limited access to healthcare facilities. The situation becomes more critical when users require immediate guidance but face barriers such as language differences, lack of awareness, or limited internet resources.

The proposed system addresses these challenges by developing an AI-driven healthcare chatbot that provides intelligent and accessible disease awareness through a web-based platform. The system integrates natural language processing techniques with machine learning models to analyze user-provided symptoms and generate meaningful responses. It utilizes SentenceTransformer-based semantic similarity to match symptoms with relevant diseases from a structured dataset, while a generative model (DistilGPT2) is incorporated as a fallback mechanism to handle ambiguous or unknown queries. The chatbot is implemented using a lightweight Flask backend, with an interactive frontend built using HTML, CSS, and JavaScript to ensure smooth user interaction.

To enhance usability and accessibility, the system supports multilingual input, allowing users to communicate in languages such as English, Hindi, Kannada, and Telugu. Additionally, voice input functionality is integrated using browser-based speech recognition, enabling users to interact with the system more naturally. The chatbot is designed to provide real-time responses with minimal latency, ensuring a seamless user experience. Security measures such as input validation and controlled API access are incorporated to maintain system reliability and prevent misuse.

Overall, the proposed solution focuses on improving accessibility, efficiency, and awareness in healthcare by providing a centralized, user-friendly platform for symptom analysis and preventive guidance. The system is technically feasible using existing AI frameworks, web technologies, and lightweight deployment models, requiring minimal resources while delivering effective performance. The chatbot has the potential to serve as a valuable tool for public health awareness, particularly in resource-constrained environments where immediate medical assistance may not be readily available.

Table of Content

Chapter No.	Title	Page No.
	Declaration	iii
	Acknowledgement	iv
	Abstract	v
	List of Figures	viii
	List of Tables	ix
	Abbreviations	x
1.	Introduction	1
	1.1 Background	1
	1.2 Statistics of Project	2
	1.3 Prior Existing Technologies	2
	1.4 Proposed Approach	3
	1.5 Objectives of the Project	4
	1.6 SDGs	4
	1.7 Overview of Project Report	5
2.	Literature Review	7
3.	Methodology	11
	3.1 Mapping of Project Phases to V-Model	11
	3.1.1 Requirement Analysis	12
	3.1.2 System Design	12
	3.1.3 Functional Design	13
	3.1.4 Unit Design	14
	3.1.5 Implementation	14
	3.1.6 Verification Phase	15
	3.1.7 Validation phase	15
	3.1.8 Testing and Evaluation	15
	3.1.9 Deployment	16

4.	Project Management	17
	4.1 Project Timeline	17
	4.2 Risk Analysis	18
	4.3 Project Budget	20
5.	Analysis and Design	22
	5.1 Requirements	22
	5.2 Block Diagram	23
	5.3 System Architecture	25
	5.4 System Flow Chart	26
	5.5 Designing Units	27
	5.6 Standards	27
	5.7 Mapping with Layers	27
	5.8 Domain Model Specification	28
	5.9 Communication Model	29
	5.10 Deployment Level	29
	5.11 Functional View	30
	5.12 Mapping Deployment with Functional View	30
	5.13 Operational View	30
	5.14 Other Design Aspects	31
6.	Hardware, Software and Simulation	32
	6.1 Hardware	32
	6.2 Software Development Tools	32
	6.3 Software Code and Implementation	33
	6.4 Simulation	34
7.	Evaluation and Results	35
	7.1 Test Points	35
	7.2 Test Plan	35
	7.3 Test Results	36
	7.4 Insights	37

8.	Social, Legal, Ethical, Sustainability and Safety Aspects	38
	8.1 Social Aspects	38
	8.2 Legal Aspects	38
	8.3 Ethical Aspects	39
	8.4 Sustainability Aspects	39
	8.5 Safety Aspects	40
9.	Conclusion	41
	References	42
	Base Paper	44
	Appendix	45
	A. Similarity Report	45
	B. Repository	45
	C. Project Output	46

List of Figures

Figure ID	Figure Caption	Page No.
Fig 1.1	Sustainable Development Goals	5
Fig 3.1	V-Model Phase mapping	12
Fig 5.1	Block Diagram	24
Fig 5.2	System Architecture	25
Fig 5.3	System Flow Chart	26
Fig A.1	Similarity Report	45
Fig B.1	Github Repository	45
Fig C.1	Home Page	46
Fig C.2	Multilingual Interface(English)	46
Fig C.3	Multilingual Interface(Hindi)	47
Fig C.4	Multilingual Interface(Kannada)	47
Fig C.5	Multilingual Interface(Telugu)	48

List of Tables

Table ID	Table Caption	Page No.
Table 1.1	Comparision of Existing Helathcare Chatbot Systems	3
Table 2.1	Summary of Literature Reviews	9
Table 4.1	Project Planning Timeline	17
Table 4.2	Project Implementation Timeline	18
Table 4.3	PESTLE Analysis	19
Table 4.4	Project Risk Matrix	19
Table 4.5	Project Budget	20
Table 5.1	Summarizing Requirement	23
Table 5.2	Mapping System layers	28
Table 7.1	Test Results of Chatbot System	36

Abbreviations

Abbreviation	**Full Form**
AI	Artificial Intelligence
API	Application Programming Interface
CSV	Comma-Separated Values
CSS	Cascading Style Sheets
D.H.U	Data Handling Unit
GPT	Generative Pre-trained Transformer
HTML	HyperText Markup Language
HTTP	HyperText Transfer protocol
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
JS	JavaScript
LLM	Large Language Model
ML	Machine Learning
PISTLE	Political, Economic, Social,Technological, Legal, Environmental
PU	Processing Unit
REST	Representational State Transfer
RNN	Recurrent Neural Network
SDGs	Sustainable Development Goals
SM	Security Module
UAT	User Acceptance Testing
UI	User Interface

Chapter 1

INTRODUCTION

Digital technologies are revamping various sectors, including health care, by speeding up information access and helping service delivery become more efficient; however, there are still lots of people who do not have prompt and accurate medical advice based on common symptoms and/or minor medical problems. Rather than finding a trustworthy and immediate source of valid medical data, most users search broadly using multiple sources on the Internet, ultimately resulting in a plethora of invalid information and conflicting opinions. A sophisticated intelligent system is needed that can provide users with organized, accurate and timely real-time health support.

Artificial Intelligence (AI) and Natural Language Processing (NLP) have led to the development of intelligent, interactive systems capable of automatically generating meaningful answers that respond to user questions. Chatbots are examples of these types of intelligent, interactive systems in which users receive support in real time by engaging in conversation. Many traditional chatbots rely on rule-based systems, which limit the bot's ability to understand the user's context and answer complicated questions. This project will develop a multilingual intelligent health care chatbot that uses AI and NLP to create a system capable of assisting in symptom identification, question comprehension and the provision of meaningful suggestions.

1.1 Background

Having access to accurate health information in a timely manner continues to be problematic for many individuals, especially in areas with limited or over-burdened healthcare service capacity. Many individuals often delay seeing a healthcare provider due to an inability to afford healthcare, lack of time, or no availability of healthcare providers. Due to this fact, many people use the internet to self-diagnose their medical condition and/or receive initial medical advice. Traditional systems that retrieve health-related data primarily rely on keyword matching, which does not necessarily provide an exact match for the original intent expressed in an individual's search. With the advancements made in machine learning, deep learning, and natural language processing (NLP), the potential exists to develop "smart" systems that can deduce from an individual’s search a more accurate semantic meaning of their input. The purpose of this project is to utilize the advances made in these technologies to create an intelligent chatbot that can assist you with your healthcare needs in a timely, reliable, accurate, and meaningful manner.

1.2 Statistics of Project

Over the last decade, there has been an increase in the uptake of digital healthcare solutions. There are also anecdotal reports that many people will first look for information about their health online before consulting their doctor. The doctor-to-patient ratio in India does not meet the standards set by global health authorities, which means that many patients do not receive prompt expert health advice.

Empirical research indicates that more than 70% of the users of online resources will search for the meaning of their symptoms and possible illnesses. Unfortunately, the majority of these resources aren't validated, and therefore, many people find it difficult to understand this information accurately. This demonstrates a need for smart systems that provide well-structured, dependable, and straightforward access to healthcare information. The introduction of artificial intelligence into healthcare applications offers the opportunity to improve access, decrease the burden on healthcare professionals, and enhance the decision-making process for healthcare providers.

1.3 Prior Existing Technologies

The different strategies developed to help users with their symptoms have been numerous in the past. The early systems were predominantly rule-based chatbots that utilized predefined rules and keyword matching techniques. While these systems perform well for simple queries, they lack the flexibility to handle complex or ambiguous user inputs.

Recent advancements in technology have introduced machine learning and natural language processing-based systems that improve the accuracy and quality of chatbot responses. These systems analyze user inputs using trained models and compare them with a knowledge base to predict possible outcomes. However, despite these improvements, many existing systems still face limitations such as lack of multilingual support, inability to handle unknown inputs, and absence of interactive features like voice input. Additionally, many of these systems require high computational resources, making them difficult to scale efficiently.

A comparison of different existing healthcare chatbot systems and their limitations is presented in Table 1.1. As shown in Table 1.1, traditional rule-based systems are simple but lack intelligence, while machine learning models improve accuracy but require large datasets. Existing health applications provide general information but lack personalization. In contrast, the proposed system uses a hybrid AI approach that combines NLP and large language models to provide accurate, multilingual, and interactive responses.

Table 1.1 Comparison of Existing Healthcare Chatbot Systems

System Type	Technology Used	Advantages	Limitations
Rule-Based Chatbots	Keyword Matching	Simple and fast	Cannot understand complex queries
NLP-Based Chatbots	Natural Language Processing	Better understanding of input	Limited to trained data
Machine Learning Models	Supervised Learning	Improved prediction accuracy	Requires large datasets
Existing Health Apps	Basic AI + Database	Provides general health information	Lack of personalization and interaction
Proposed System	Hybrid AI (NLP + LLM)	Accurate, multilingual, interactive	Depends on dataset quality

1.4 Proposed Approach

Project Objective

This project aims to create and implement an intelligent, AI-driven, multilingual healthcare chatbot that can analyse users' symptoms, accurately estimate what illness they might have, and provide preventative advice in an efficient and user-friendly manner.

Rationale

We want to provide users with a speedy, reliable, easily accessible resource to act as their first point of contact for basic medical advice, thereby reducing reliance upon unreliable internet sources and improving the quality of responses they receive through advanced AI techniques.

Proposed Methodology

This hybrid AI solution will use both semantic similarity approaches along with generative AI models. Initially, user inputs will be processed through a language detection module to determine the user's input language. Next, the user input will be transformed into vector embeddings using a sentence transformer model, which allows for semantic representation of symptoms.

These embeddings will then be computed for their cosine similarity to a pre-existing dataset of corresponding diseases, with the highest ranked disease retrieved and displayed — along with its associated preventative care tips. In instances where a disease cannot found by comparing to existing data, DistilGPT2 will be utilized to create meaningful and contextually relevant

responses**.** Through incorporating browser-based speech recognition, this design also allows the user to interact with the system via voice commands. Additionally, the complete system is designed using the client-server architecture with Flask performing all back-end and front-end communication

Some applications for the Project include:

- Providing an initial diagnosis from the users' reported symptoms
- Providing education to users regarding various health issues/topics
- Providing an educational tool to help increase awareness of health issues
- Providing a service to users in remote or underserved areas

Limitations of the Proposed Approach

- The proposed system will not replace professional medical advice/diagnosis
- The accuracy of predictions will depend on the size and quality of the underlying dataset
- The system may not be able to provide adequate results for unusual or complex medical conditions
- An internet connection is required for the system to function as designed

1.5 Objectives of the Project

The objectives of the project are as follows:

- Develop a system that will process user input to provide a healthcare response.
- Apply semantic analysis methods for interpreting symptoms.
- Implement generative AI models to respond to unknown and unmatched queries.
- Provide secure communications and data handling within the system.
- Create a scalable and easy-to-use web-based application that offers multilingual support.

1.6 SDGs (Sustainable Development Goals)

The objectives of the United Nations Sustainable Development Goals (SDGs), as illustrated in Fig 1.1 (United Nations, 2015), are aligned with this project as follows:

- **Good Health and Well-being (SDG 3):** The system provides basic healthcare awareness by predicting diseases based on symptoms and suggesting preventive measures.

- **Industry, Innovation and Infrastructure (SDG 9):** The project utilizes advanced technologies such as Artificial Intelligence and Natural Language Processing to develop an innovative and scalable healthcare solution.
- **Reduced Inequalities (SDG 10):** The chatbot supports multiple languages and voice input, enabling accessibility for users from different regions and backgrounds.

Fig 1.1 Sustainable Development Goals

1.7 Overview of the Project Report

The report will be separated into several chapters, each addressing one of the various aspects of the project.

- The first chapter provides an overview of the project, including its background, objectives, and an overview of how it was successfully accomplished.
- Chapter two includes a comprehensive literature review of different existing systems and technologies that have been developed in the area..
- The third chapter outlines the methods that were used in developing the system.
- Chapter four describes the different aspects of project management concerning the project, including planning of the project, risk analysis and potential risks, and budgeting for the project.
- The fifth chapter describes the techniques that were utilized in analyzing the system and designing it, including the architecture and the functional modules.
- The sixth chapter details the tools that were used in implementing the software.
- The seventh chapter provides the evaluation of the results from the system's tests and the system's performance if applicable.

- The eighth chapter provides a discussion about the social and legal, ethical, and sustainability implications of the project.
- The ninth chapter of the report provides a conclusion and makes suggestions as to how the project could be improved in the future.

Chapter 2

LITERATURE REVIEW

Recently, there has been an increased focus on the creation of intelligent health care systems employing Artificial Intelligence (AI) or Natural Language Processing (NLP), and many researchers have been investigating various methods to make chatbot systems available to help users discover disease states and receive medical assistance.

Sharma et al. designed an AI-based healthcare chatbot using NLP techniques to read user-completed queries and generate responses based on pre-defined medical data sets. The chatbot uses a symptom-based approach to predict disease states and then offers basic treatment suggestions. The results of this study demonstrate an overall improvement in accuracy when compared with traditional keyword-based chatbots; however, due to its reliance on structured data sets for answering questions, the bot cannot respond effectively to complex/ambiguous queries .

The disease prediction model that Kumar and Singh designed utilizes machine learning techniques (specifically supervised learning algorithms), such as decision trees and random forests, to predict disease states based upon the symptoms reported by users (i.e., the symptoms that they provide in their queries). Overall there was a significant improvement in terms of predicted accuracy when the use of these technologies was analyzed; however, it should also be noted that these models need a high number of training examples to function effectively, and therefore there are limitations on how interactive the user experiences are when using them (e.g., limited ability for real-time conversations).

Using recurrent neural networks (RNN), Patel et al. have created a deep learning-based chatbot that uses deep learning to understand user input better than traditional chatbots by utilizing RNN for better context to understand the specific inputs. The RNN chatbot also improved sequential data processing to allow for a better flow of conversation for both parties. Even though this system's capabilities far exceed the capabilities of traditional chatbots, it still requires large amounts of processing power; therefore, it is not practical for use in real-time environments with limited resources.

Another example of a medical chatbot is one that was developed by Reddy and Kumar. Their system provides users' health information in response to static conditions and pre-defined responses. Because the system uses predetermined data to respond to queries, it is an efficient solution for basic question-and-answer scenarios. However, this system lacks the flexibility to

adapt to any questions that are not previously known or present to the medical database, which limits the usability of the system to real-world conditions that are always changing.

Gupta et al. have created a medical chatbot utilizing similarity-based semantics by creating vector embeddings of sentences from known ontology, allowing for the matching of the user's input to an appropriate medical dataset by using the cosine function to determine the best fit. By implementing cosine similarity for semantic matching instead of utilizing keyword matching per the typical methods used by other previous chatbots, Gupta et al. improved the overall accuracy of predicting disease based on the users' intent. However, chatbot performance for Gupta et al. is heavily dependent upon both the accuracy of the vector embeddings as well as the completeness of the underlying dataset.

Li et al. developed a proposed conversational health model based on transformers for use in the healthcare environment, leveraging the benefits associated with response generation with previously trained language models to bring a more human-like response to complex user queries in addition to using the correct language response or format method to produce a completion for user input. Findings indicate that conversational abilities and answer quality are improved through the chatbot. Due to the limited or one-sided sources of training data, however, there is a potential for incorrect or misleading response generation by the model as a result of this data shortage.

Ahmed et al. created a multilingual chatbot framework that utilizes language identification methods to accommodate diverse audiences. In addition to expanding access to users with different backgrounds and improving user experience, maintaining an accurate representation across the various languages of the system continues to pose a problem because of the differences in grammar, word structure, and how the same word may be handled differently within the various datasets.

Singh et al.'s work involved developing a voice-driven chatbot application using voice recognition technology. A voice-based input would then be transcribed to text, and the chatbot would generate a response(s) based upon the input. This would enhance usability for users with limited technological expertise. However, the performance of the voice recognition technology is subject to interference from ambient/background noise, speech patterns, and pronunciation.

Zhang et al. propose a hybrid artificial intelligence technique combining both rule-based and machine learning (ML) methodologies to create an application that can efficiently and effectively provide healthcare services. By employing a rule-based approach for handling queries based on available knowledge and an ML-based approach for responding to queries that cannot be answered based on previous information, this application achieves flexibility

and higher accuracy. In addition, the complexity of integrating these technologies and maintaining them continues to challenge the viability of this application.

Throughout the literature examined, there appears to be substantial advancements in creating a healthcare delivery chatbot. However, the barriers identified were related to the limited ability for semantic understanding, operating in multiple languages, and relying upon different forms of data as sources of information; specifically, responses to an unknown query would also be difficult to create or generate with these limitations. The existing gaps serve as motivation for the design and implementation of a new type of hybrid AI-based multilingual chatbot that utilizes semantic similarity techniques and generative models to enhance production accuracy, increasing user interaction and improving overall user experience.

Table 2.1 Summary of Literature Reviews

Author	**Approach Used**	**Key Contribution**	**Limitation**
Sharma et al.	NLP-based chatbot	Symptom-based response system	Limited handling of complex queries
Kumar & Singh	Machine Learning	Disease prediction using ML	Requires large dataset
Patel et al.	Deep Learning (RNN)	Context-aware conversation	High computational cost
Reddy & Kumar	Rule-based system	Simple and fast responses	No adaptability
Gupta et al.	Semantic similarity	Improved intent understanding	Dataset dependent
Li et al.	Transformer model	Human-like responses	Risk of incorrect output
Ahmed et al.	Multilingual chatbot	Language support	Accuracy issues in multiple languages
Singh et al.	Voice-based system	Speech interaction	Noise sensitivity
Zhang et al.	Hybrid AI model	Flexible response system	Complex implementation

Table 2.1 depicts the different methodologies and constraints of the prominent approach to SLG identified from previous literature. Rule-based and machine learning methods dominate the current reconstruction process and are the foundations for the majority of the systems; thus, they both afford a certain amount of flexibility and accuracy. Therefore, the author proposes an integrated approach to resolve the primary limitations of those two methodologies by merging generative AI models with traditional semantic similarity strategies to enhance user interaction while providing a more flexible and accurate result overall.

Chapter 3

METHODOLOGY

To develop an artificial intelligence based multi-lingual health care chatbot, a systematic plan with structure is needed so that the application can be both reliable and scalable as well as accurate. As it pertains to software engineering, the manner in which a development methodology is selected is critical in achieving both reliability, scalability and accuracy for this project. The use of V-model methodology was used as the methodology chosen because of the clear connection between the development phases of the life cycle to that of the corresponding testing phases for verification and validation at each phase of the life cycle of the system being developed.

In its simplest form, the V-model is an extension of the waterfall model; in that for each phase of development there is a corresponding phase of testing. This technique provides for errors to be found at the earliest possible point during the life cycle of the developed software (therefore minimizing development cost while providing a higher quality end product). Additionally, for the purposes of this project, the V-model is particularly applicable given that this type of system will be comprised of multiple components which consist of multiple front-end interfaces, multiple back-end APIs, AI models, data processing modules, all of which will be required to undergo thorough testing for validation.

The V-model is provided in figures 3.1 The left half (the verification phase) of the V-model is comprised of three separate phases, which include requirement analysis, system design, and module design. The right half (validation phase) of the V-model includes four separate phases that include unit testing, integration testing, system testing and user acceptance testing. By having the clear and structured mapping of the V model, we ensure that every development activity will be validated through the corresponding testing activities associated with it.

3.1 Mapping of Project Phases to the V-Model

The V-Model for ScanWise consists of the following phases: requirement analysis, system design, functional design, unit design and implementation, unit testing, integration testing, verification, and validation. Each phase corresponds to a distinct stage in building the multimodal clinical decision support framework.

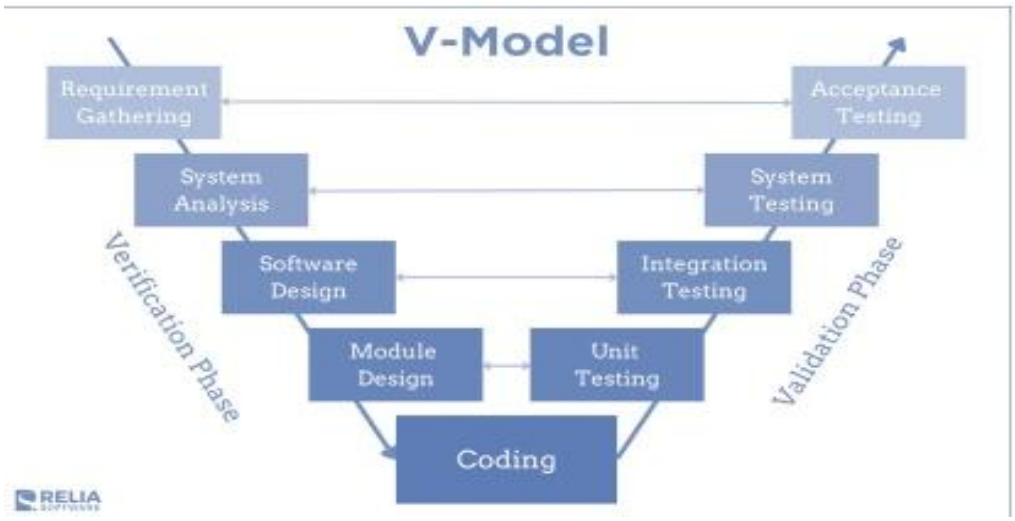

Fig 3.1 V-Model phase mapping

3.1.1 Requirement Analysis (Specification Phase)

In the requirement analysis stage of healthcare chatbot development, functional and non-functional requirements will be identified, documented, and derived from user needs, system limitations, and literature review. As the basis of the system being developed, the functional requirement for the chatbot includes the ability to receive input from users (symptoms) and return disease predictions along with preventive recommendations. Multilingual input (English, Hindi, Kannada, and Telugu) will enable users to interact with the system in different languages. The chatbot will also accept both voice and text inputs, allowing the system to provide a more accessible experience for users. Non-Functional Requirements include performance, scalability, security, and usability. Users should experience a quick response time; thus, the system will need to perform quickly. Users also want a scalable product; therefore, the system will need to handle multiple users at the same time. Security is critical to prevent unwanted (malicious) input and to ensure the safe processing of user data. In this phase of the project, any constraints (i.e., data dependency, AI model limitations, and system resource requirements) will be identified and documented so that clear requirements are developed at the initial phase of development to provide a clearly-defined path for development.

3.1.2 System Design

The design phase of the system involves an overview of what the architecture of the chatbot system will look like. The proposed solution will use a client-server architecture to allow the backend and frontend parts of the system to communicate via RESTful APIs.

For this project, the frontend is responsible for interacting with users, which will be implemented using HTML, CSS, and Javascript. The frontend will offer an easy-to-use interface where users can input their symptoms by either writing or talking about them.

The backend is built on top of the Flask web framework and is responsible for processing requests from the client, communicating with AI models, and generating responses.

The design phase also outlines several modules that are part of this system: input processing, language detection, semantic analysis, dataset matching, and response generation. Each of these modules has an individual function and can be updated without impacting any of the other modules (modular design).

The data flow through the system has been thoughtfully designed for maximum efficiency. Therefore, once a user submits his or her symptoms, the raw input is first processed to remove unwanted characters (cleaned). The next step is for the language detection module to identify the language of the user input, at which point the text will be converted into vector embeddings based on a sentence-transformer model. As the next step, we will compare the generated embeddings with feasible syntactic datasets using cosine similarity until we identify the most similar entry (closest match). However, if there is no match, we will utilize a generative AI model to create a response.

3.1.3 Functional Design

In this phase of developing the functional design, the system is split into many smaller pieces, and this modularity will allow for easier development and testing and also improve the reliability of the entire system.

The following are the major functional modules that the complete system contains:

1. The User Interface Module - This module provides the user interface for the chatbot, handles interaction with users, and presents the output from the chatbot to the user. The user interface has been designed to be responsive and usable from many different devices.
2. The Backend Processing Module - This module will manage API requests and the communications of the various components of the system. This module will also serve as the central controller for the operation of the entire system.
3. The AI Processing Module - This module will analyze user input using semantic similarity and provide a response generated by one or more AI models. The AI Processing module is a critical part of the system and ensures that the responses provided by the chatbot are accurate and meaningful.

4. The Data Processing Module - This module will handle all operations on the data set, such as reading, filtering, and extracting information from the data set. This module will help ensure that data is efficiently handled and retrieved.
5. The Security Module - The security module is responsible for allowing safe processing of user input by preventing the execution of malicious scripts or limiting the ability to make excessive API requests. This makes the system more reliable and provides protection from potential attacks.

3.1.4 Unit Design (Software Components)

In the unit design phase, designers create and implement individual modules of the system. With this kind of development process you are able to perform your debugging processes and develop without distraction from other parts of the system.
The front end unit of the system was designed to allow users to provide input and to provide dynamic displays of chatbot responses. Additionally, the front end was designed to handle voice input through the use of built-in speech recognition capabilities of web browsers.
The back end unit is responsible for processing and integrating the end users' requests and managing the AI model. The back end handles all of the routing of requests, handles the requests, and produces the responses.
The AI unit processes users input using sentence transformer models to create representations (embeddings) for each input and generate a similarity score. Also, the AI uses a generative-based AI model (DistilGPT2) to manage any unmatched queries.
The data unit of the system manages the data related to the disease and stores it in a CSV format for ease of retrieval and filtering.
Each unit is thoroughly tested for functionality prior to integration into a whole system.

3.1.5 Implementation

Building the system according to its design specifications is the first step in the implementation phase. The front-end of the system will be built using HTML, CSS and JavaScript so that a clean and interactive interface can be provided to users. The back-end of the system will use Flask for processing API requests and combining its components.
The AI models for semantically similar sentences will use Sentence Transformers to complete this task, while Hugging Face's Transformers will be used to generate fallback responses.
The dataset is processed via Pandas, while any numeric computations are processed using NumPy.

The implementation phase will also involve integrating multiple modules so that they can successfully communicate with and work properly together; provisions will be made for proper handling of errors such as unexpected input or system failure.

3.1.6 Verification Phase

The purpose of the verification phase is to confirm that every step of the development process satisfies the appropriate specifications. The verification phase concentrates on the examination and validation of deciding to design, and implement.

Requirement verification is to ensure that every user requirement was defined clearly and addressed in a correct manner. Through design verification, we check whether the design meets the specified requirements of the system architecture. Module verification ensures that every component of the model functions correctly as specified by its design.

It also supports the identification of errors at an early stage in the development process, thus minimizing the likelihood of defects occurring at a later stage.

3.1.7 Validation Phase

The validation phase guarantees that the build meets expectations and operates as anticipated. In this phase of development, a number of levels of testing will be conducted.

Individual units of a system are tested to verify individual units of development, such as the success of a language detection unit, a similarity match unit, and a response generation unit (for both individual and all three), respectively, via:

- unit testing
- integration testing
- system testing

You will perform different types of testing to evaluate the overall performance of the entire system under many different scenarios.

User acceptance testing (UAT) is conducted to verify that the user has received what they expected and the system meets the user's requirements and expectations. In support of UAT, user feedback is used to enhance the performance of the system and improve its usability.

3.1.8 Testing and Evaluation

Testing of the system is performed under multiple circumstances to determine how well it performs and operates.

Test cases are created to verify disease prediction accuracy and response generation accuracy. The performance of the system is evaluated on the basis of such criteria as accuracy, response time and user satisfaction. The multilingual capability was also tested to ensure that language detection and response generation are conducted accurately, and the voice input feature was tested to verify the accuracy of the system's voice-activated features under a variety of conditions.

The results of the evaluation will provide insightful information on whether or not the system meets expectations and opportunities for further development or improvements.

3.1.9 Deployment

In order to provide the users with access to a functional system, the deployment stage must take place. The chatbot will be accessible as a Web Application through a standard browser.

In the backend, all functions of the chatbot will run on a Flask Server, & its front-end will provide an interactive user interface.

The system has been designed for scale, allowing for many registered users to use the chatbot at once without experiencing performance issues.

As all configuration settings have been defined and tested before deployment, both the stability of the system, as well as its performance, will be reliable during use.

Chapter 4

PROJECT MANAGEMENT

4.1 Project Timeline

The process of project management plays a crucial role in the successful development of an AI-driven healthcare chatbot. A well-defined project timeline helps in organizing tasks efficiently, allocating resources appropriately, and monitoring the progress throughout the development lifecycle.

The project was executed based on a structured timeline divided into two main phases: planning and implementation. A Gantt chart is used to visually represent the sequence of tasks, their duration, and dependencies. This approach helps in identifying milestones and ensures that the project progresses according to the planned schedule. The use of a Gantt chart also supports effective time management and minimizes delays during development.

The key activities included in the project timeline are requirement analysis, literature review, system design, development, testing, and deployment. Each task is scheduled carefully to ensure smooth execution without affecting the overall progress of the project.

The detailed project planning timeline is presented in Table 4.1. As shown in Table 4.1, the initial phase of the project focuses on problem identification, literature review, and requirement analysis, followed by project planning activities. This structured approach ensures that all foundational aspects of the system are clearly defined before moving to the implementation phase.

Table 4.1 – Project Planning Timeline

Task	**Description**	**Duration**	**Start Week**	**End Week**
Problem Identification	Define project problem and scope	1 week	Week 1	Week 1
Literature Review	Study existing systems and research papers	2 weeks	Week 2	Week 3
Requirement Analysis	Identify system requirements	1 week	Week 4	Week 4
Project Planning	Define timeline and resources	1 week	Week 5	Week 5

Table 4.2 Project Implementation Timeline

Task	**Description**	**Duration**	**Start Week**	**End Week**
System Design	Design architecture and modules	1 week	Week 6	Week 6
Frontend Development	Develop UI using HTML, CSS, JS	2 weeks	Week 7	Week 8
Backend Development	Develop Flask API	2 weeks	Week 9	Week 10
AI Model Integration	Integrate NLP and GPT models	2 weeks	Week 11	Week 12
Testing	Perform unit and system testing	1 week	Week 13	Week 13
Deployment	Deploy application	1 week	Week 14	Week 14

The implementation phase of the project focuses on the actual development and deployment of the system. This phase includes designing the system architecture, developing the frontend and backend components, integrating AI models, and performing testing and deployment activities. The detailed implementation timeline is presented in Table 4.2. As shown in Table 4.2, the implementation phase begins with system design, followed by frontend and backend development. The integration of AI models is carried out after the core system development, ensuring that the chatbot functionalities are properly embedded. The final stages include testing and deployment, which ensure that the system operates correctly and is ready for real-time usage.

This structured implementation approach ensures that each stage of development is completed systematically, reducing errors and improving the overall efficiency of the project.

4.2 Risk Analysis

Risk analysis is an important part of project management as it helps in identifying potential risks and developing appropriate mitigation strategies to ensure successful project completion. In this project, a PESTLE analysis is conducted to evaluate the external factors that may influence the development and deployment of the AI-driven healthcare chatbot system.

The components of the PESTLE analysis include political, economic, social, technological, legal, and environmental factors. Political factors involve regulations related to healthcare data and AI usage. Economic factors focus on the cost of development and resource allocation. Social factors consider user acceptance and awareness of AI-based systems. Technological

factors relate to system reliability, scalability, and performance. Legal factors address data privacy and compliance requirements, while environmental factors consider energy consumption and system efficiency.

The detailed PESTLE analysis for the project is presented in Table 4.3. As shown in Table 4.3, each factor is associated with specific risks and corresponding mitigation strategies. For example, data privacy risks are addressed by avoiding the storage of sensitive information, while budget constraints are managed through the use of open-source technologies. This structured analysis helps in identifying critical challenges and ensures that appropriate measures are taken to minimize their impact on the project.

Table 4.3 PESTLE Analysis

Factor	**Risk**	**Mitigation**
Political	Regulations on health data	Follow compliance guidelines
Economic	Budget constraints	Use open-source tools
Social	Low user trust	Provide accurate responses
Technological	System failure	Use reliable frameworks
Legal	Data privacy issues	Avoid storing sensitive data
Environmental	Energy usage	Optimize system performance

Table 4.4 Project Risk Matrix

Risk	**Probability**	**Impact**	**Mitigation**
Incorrect prediction	Medium	High	Improve dataset quality
System crash	Low	High	Error handling
API misuse	Medium	Medium	Rate limiting
Poor performance	Medium	Medium	Optimize models

Risk assessment plays a crucial role in identifying potential challenges before the implementation of the project. By analyzing risks at an early stage, appropriate mitigation strategies can be developed to ensure smooth execution and successful project delivery.

The project risk matrix is presented in Table 4.4. As shown in Table 4.4, different risks are evaluated based on their probability and impact. For instance, incorrect disease prediction is

considered a high-impact risk and is addressed by improving the quality of the dataset. Similarly, system crashes are mitigated through proper error handling mechanisms, while API misuse is controlled using rate limiting techniques. Performance-related risks are managed by optimizing the AI models used in the system.

This structured risk assessment approach helps in prioritizing critical risks and implementing effective mitigation strategies, thereby improving the overall reliability and performance of the system.

4.3 Project Budget

The overall budget for a project is an essential component of managing the project, as it will provide information on the estimated costs associated with developing and deploying the scope of work. Since most tools and technologies used in this project are open-source, there is a significant reduction in overall costs. The project budget includes cost associated with all of the following development tools, computing capabilities, and infrastructure required for deployment. Because of the use of open-source frameworks (i.e., Flask, Python libraries, browser-based APIs), the total costs to develop this project remain low. The overall budget for the project will be based upon the required tasks, the availability of the resources to perform the tasks, and the expected length of time required to develop. Properly budgeting for each step of your project will ensure that you use your resources economically and will help to avoid any unnecessary expenditures.

Table 4.5 Project Budget

Resource	**Description**	**Cost**
Software Tools	Python, Flask, VS Code	Free
AI Models	Sentence Transformers, GPT	Free
Dataset	CSV dataset	Free
Hosting	Localhost / Cloud (optional)	Minimal
Miscellaneous	Internet, electricity	Low

The project budget provides an overview of the resources required for the development and deployment of the system. It highlights the cost efficiency of the proposed solution by utilizing freely available tools and technologies.

The detailed project budget is presented in Table 4.5. As shown in Table 4.5, most of the resources used in the project, including software tools such as Python, Flask, and Visual Studio Code, as well as AI models like Sentence Transformers and GPT, are open-source and freely available. The dataset used for training is also in CSV format and does not incur any cost. The hosting requirements are minimal, as the system can be deployed locally or on low-cost cloud platforms. Additional expenses such as internet and electricity are also relatively low.

This demonstrates that the proposed system is cost-effective and feasible, making it suitable for large-scale adoption without significant financial investment.

Chapter 5

ANALYSIS AND DESIGN

5.1 Requirements

A second important phase of the development of an AI-based healthcare chatbot is the analysis phase. The analysis phase is used to determine what the requirements are for building the system, and what the area is that the proposed system will operate in, i.e. the problem domain. The proposed AI-based healthcare chatbot will process user symptoms (input) and provide predictions for diseases, along with suggestions for preventing the disease.

The main purpose of the proposed system is to assist users in acquiring reliable and structured access to preliminary healthcare information. The behaviour of the system includes the acceptance of user input, processing of user input using AI models, and the generation of appropriate responses to the user. The AI-based healthcare chatbot will also support multilingual input and voice commands to facilitate accessibility for all users.

The requirements for the AI-based healthcare chatbot can be divided into functional and non-functional requirements. The functional requirements for the AI-based healthcare chatbot include the analysis of user symptoms, the prediction of diseases, support for multiple languages, and the generation of responses to users. The non-functional requirements for the AI-based healthcare chatbot include the performance, scalability, usability, and security of the chatbot.

The input for the AI-based healthcare chatbot will be user symptom information and will be processed using AI techniques. The output of the AI-based healthcare chatbot will be a prediction of a disease and suggestions for preventing or treating it. Some of the constraints that will impact the development of the AI-based healthcare chatbot will be the quality of the dataset that is used to build the AI models for the chatbot, as well as the limitations of the AI models themselves.

Data collection requirements for the AI-based healthcare chatbot will include storing disease-related data in a structured format (e.g. CSV files). The data analysis requirements for the AI-based healthcare chatbot will include comparing user input to dataset entries using semantic similarity techniques and performing all processing in real-time.Table

5.1 Summarizing Requirements

Category	Description
Purpose	AI chatbot for disease prediction using symptoms
Behaviour	Accept input → Process → Predict disease → Show response
System Management	Handle API requests and manage modules
Data Analysis	Perform semantic similarity using embeddings
Deployment	Web-based application accessible via browser
Security	Input validation, rate limiting

The system requirements define the functional and operational aspects necessary for the successful development of the AI-driven healthcare chatbot. These requirements help in guiding the design, implementation, and evaluation of the system.

The summarized system requirements are presented in Table 5.1. As shown in Table 5.1, the primary purpose of the system is to provide disease prediction based on user-input symptoms. The system behaviour involves accepting user input, processing it using AI models, predicting the possible disease, and displaying an appropriate response.

The system management component ensures proper handling of API requests and coordination between different modules of the application. Data analysis is performed using semantic similarity techniques based on embeddings to improve prediction accuracy. The system is designed as a web-based application, allowing users to access it easily through a browser.

Security requirements are also considered, including input validation and rate limiting to prevent misuse and ensure safe operation of the system. These requirements collectively ensure that the system is efficient, reliable, and scalable.

5.2 Block Diagram

Fig 5.1 shows the functional block diagram of the proposed AI-driven healthcare chatbot system. The diagram provides a high-level representation of how different components of the system interact to process user input and generate responses.

The system is divided into three main blocks: input, processing, and output. The input block allows users to interact with the chatbot using either text-based input through a keyboard or voice-based input through a microphone. This ensures ease of use and accessibility for different types of users.

The processing block is the core of the system and consists of multiple modules responsible for analyzing the input and generating responses. The language detection module identifies the language of the user input, enabling multilingual support. The semantic analysis module converts the input into meaningful representations using embedding techniques. The dataset matching module compares the processed input with stored disease information using similarity measures. If no suitable match is found, the response generation module uses a generative AI model to produce an appropriate response.

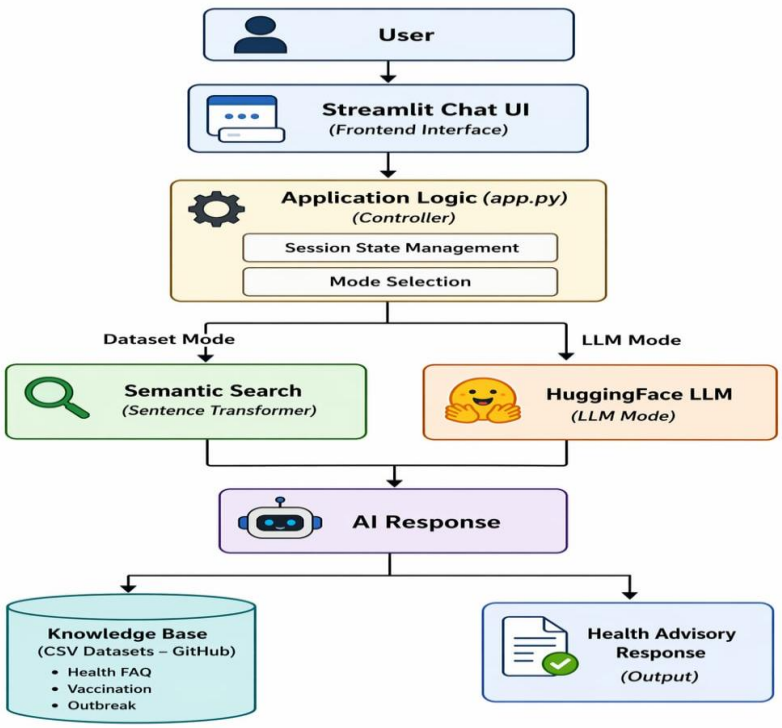

Fig 5.1 Block Diagram

The output block presents the final response to the user in a readable format. The response may include predicted disease information and preventive measures. The system ensures real-time interaction and accurate output generation.

Overall, the functional block diagram provides a clear visualization of data flow within the system and helps in understanding the interaction between various components involved in chatbot operation.

5.3 System Architecture

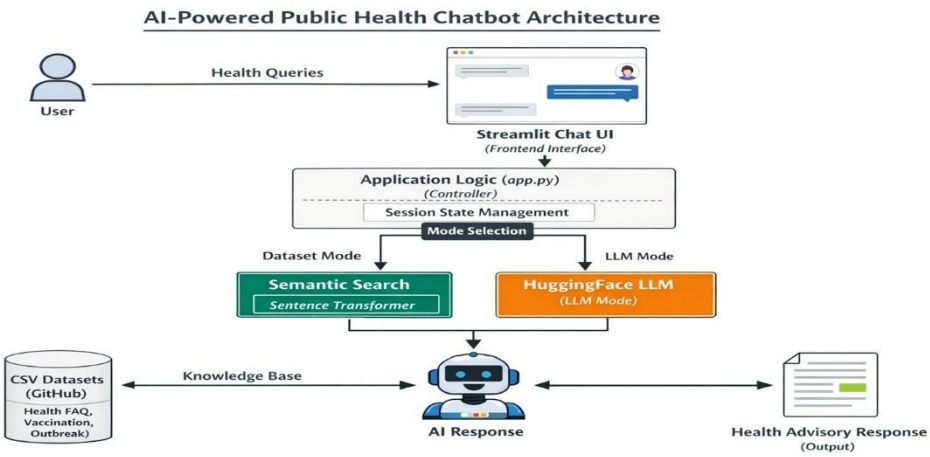

Fig 5.2 System Architecture

The AI-Powered Public Health Chatbot Architecture (as shown in the fig 5.2) is designed to deliver accurate and intelligent health-related responses through a hybrid approach. The process begins when the user submits a query via the Streamlit Chat UI, which acts as the frontend interface. This input is then passed to the Application Logic (app.py), functioning as the controller layer. Here, session state management maintains conversation context, and a mode selection mechanism determines how the query should be processed. The system intelligently routes the request either to Dataset Mode or LLM Mode based on the type and complexity of the query.

In Dataset Mode, the system uses Semantic Search with Sentence Transformers to retrieve relevant information from the CSV-based knowledge base (GitHub datasets) containing health FAQs, vaccination data, and outbreak details. In contrast, LLM Mode utilizes a HuggingFace Large Language Model to generate context-aware and conversational responses for complex queries. Both modes contribute to the AI Response module, which formats and delivers a final Health Advisory Response back to the user. As illustrated in the diagram, this architecture ensures a balance between factual accuracy and dynamic response generation, making it effective for real-time public health assistance.

5.4 System Flow Chart

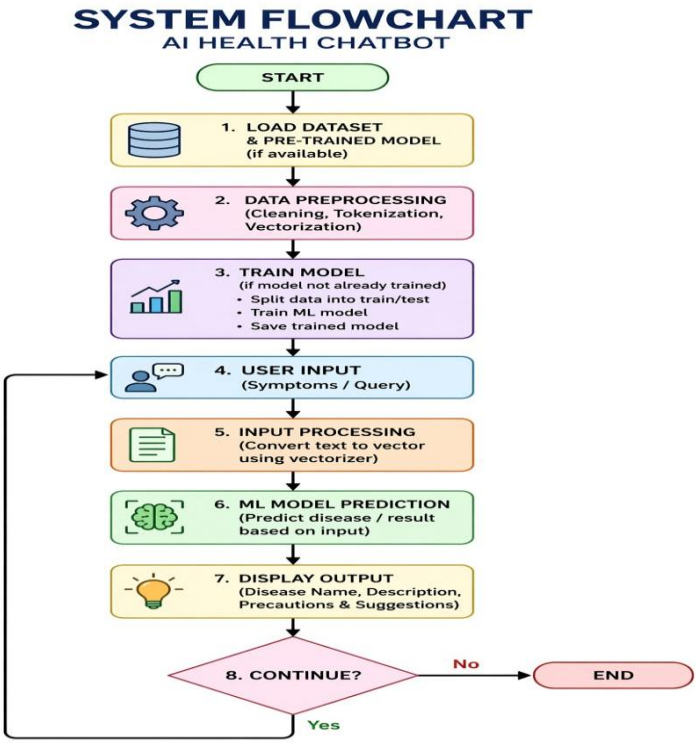

Fig 5.3 System Flow chart

The overall working of the chatbot system is illustrated in Figure: System Flowchart of AI Health Chatbot. As shown in the figure, the process begins with the user input stage (Step 4), where the user provides symptoms or a query to the system. This input is then passed to the input processing stage (Step 5), where preprocessing operations such as text cleaning, tokenization, and vectorization are performed.

Following preprocessing, the system converts the input into an embedding format and compares it with the entries in the dataset using similarity techniques. This process is represented through the ML model prediction stage (Step 6) in the flowchart. If a match is found in the dataset, the chatbot retrieves the corresponding disease information and associated preventive measures. The results are then displayed to the user in the output stage (Step 7), which includes the disease name, description, and suggestions.

If no similar entry is found in the dataset, the system utilizes a generative AI model to produce an appropriate response. This ensures that the chatbot can handle unseen or unknown queries effectively. Finally, as indicated in the decision block (Step 8) of the flowchart, the system checks whether the user wants to continue. If yes, the process repeats from the input stage; otherwise, the system terminates.

Thus, the flowchart provides a clear representation of the logical sequence of operations, helping developers understand the system workflow and ensuring proper execution of each stage

5.5 Designing Units

In designing the system, it was organized into a number of separate units to simplify the development process and to increase the modularity of the system. Each unit of the system carries out its own function and interacts with other units of the system to provide a completed system function. The user input unit will allow a user to provide input via either text or voice. The processing unit will include both the functions for detecting the language of the input, generating the embedding of the input and matching the input with the entries in the data set for similarity. The AI unit will generate the response for the user using a generative pre-trained transformer (GPT) model. The output unit will display the response to the user. Each unit is designed and tested independently, and once testing has been completed, they are integrated into the entire system. This approach enables the maintainability and scalability of the system.

5.6 Standards

The system utilizes several types of standards for the purpose of achieving cross-component interoperability, security and efficient communication between system components.

API communication from the front-end application layer to the back-end application layer is accomplished by using REST APIs with HTTP protocol as the transport layer protocol. Data exchange is done using JSON format, which is lightweight and easy to process.

A set of standards exist to guarantee that the software operates securely, including input checking and sanitising to mitigate against malicious attacks. The software will also implement a rate limiting functionality to limit the use of each individual API to prevent overwhelming the server with excessive API requests. Front-end application development will be accomplished using standard web technologies such as HTML, CSS, and JavaScript.

Through the implementation of these standards; reliable, secure and efficient operation of the system will be possible.

5.7 Mapping with Layers (Adapted for Software System)

Modular, scalable, and efficient operations are made possible through the proposed AI-based health chatbot system using a layered approach. A series of distinct functional layers are

incorporated to provide isolation of concerns which increases maintainability and improves security.

The user interface of the application layer creates the visual elements of the application using HTML, CSS, & JavaScript, allowing it to communicate with users, collect symptom inputs, and display responses from the chatbot. Input validation is performed at the application layer before any values are processed through the system, eliminating invalid or malicious inputs.

The processing layer is built on top of the Flask web framework as the backend for handling all API requests. The processing layer processes user input, performs the necessary processing, and coordinates communication between different components of the system. In conjunction with the API rate-limiting and CORS policies, the processing layer provides a secure environment to provide controlled access.

The AI layer contains all the components and processing required to provide natural language understanding and response generation, including a SentenceTransformer model for semantic similarity matching and a DistilGPT2 model for generating fallback responses. The AI layer is responsible for all intelligent decision-making based upon user input. The data source consists of our data which is stored in CSV format include information on diseases; symptoms and ways to prevent disease spread. Data accuracy and consistency are taken into account to ensure our predicted information is as correct as possible.

Table 5.2 Mapping System Layers

Layer	**System Component**	**Security**
Application Layer	UI (HTML, JS)	Input validation
Processing Layer	Flask Backend	API security
AI Layer	NLP + GPT Models	Controlled responses
Data Layer	Dataset (CSV)	Data integrity

Table 5.2 shows how the various layers of the system are mapped against their corresponding components and security methods. This layered architecture allows multiple concerns to be separated and also increases both reliability & scalability for the overall system.

5.8 Domain Model Specification

The Domain Model describes the primary Entities of the System along with their relationships and provides a conceptual representation of how the various components of a Healthcare

Chatbot interact together. The four main entities include the User, Symptoms, Disease, and Responses to those diseases. As a User provides symptom-based input to the System, the AI Model will process that symptom-related input and will return the most relevant Disease Entity from the database. Each Disease Entity contains a collection of Symptom Entities, along with their Associated Preventive Measures. The Response Entity contains the output provided by the System which consists of information predicting what Disease the User may possibly be suffering from, along with Suggested Preventive Measures. The relationship structure among these Entities has been designed in a way to allow the User Input, AI Model processed Input, and meaningful response to be generated from the System. The relationships have also been designed in a way that facilitates a clear Path of Data through the System, which will assist in System Design Efficiencies. The Domain Model can be used to provide a better understanding of how the System behaves and will be beneficial for additional future improvements and for the scalability of the System.

5.9 Communication Model

Using a request-response communication model—common across web applications—one website manages service delivery and two staff members can interact using one mobile telephone to access its services. The frontend (user) submits a symptom(s) to the website through the chatbot interface. This request is sent through to the Flask (backend) application via HTTP requesting symptom processing through NLP models and looking up relevant records within the project dataset(s). Backend responds back to the front-end (user) with the symptom(s) processing result returned through and utilizing an HTTP response. Project appropriateness: Using this comms model provides a simple, reliable and real-time interaction mechanism as well as allows for easy integration between other systems and scalability of handling multiple users concurrently.

5.10 Deployment Level

The system, which is proposed here, is going to be available as an online application via the use of the internet. The frontend will be served live in the user's web browser and the backend will be hosted on our web server and built on the Flask framework. The benefit of this type of deployment model is that the user has the capability to access the application using any web-enabled device, completely eliminating the need for installation. Additionally, by utilizing a cloud-hosted server model, it is possible to conduct centralized management of the data and the underlying AI models. The proposed system can be deployed and used locally for

development purposes and or on cloud-based servers for large scale use, making it a highly scalable and adaptable system capable of being used in a wide variety of locations and for a wide variety of purposes.

5.11 Functional View

The system's functional perspective outlines all the various functional Componentry's roles as part of an operation. The system is divided into (5) five Functional Units; User Interface (UI), Processing Unit (PU), Artificial Intelligence (AI) Module, Data Handling Unit (D.H.U), and Security Module (SM). The UI enables capturing of user input and providing responses. The PU manages the data flow between all other components of the System. The AI Module provides Syntactic Analysis and generates answers, or responds, to User Input. The D.H.U manages all operations associated with DataSets. The SM ensures that the overall system operates securely. All (5) functional units will independently perform their duty while providing on-going collaboration between the different functional Units in order to support the System operating in a continuous manner.

5.12 Mapping Deployment with Functional View

To develop an efficient system, it is necessary to provide a clear mapping of how each layer in the deployment architecture maps to a functional component within reason. Each layer's deployment architecture will correspond to either an interface for end users (i.e., user interface) or a component that processes data, operates AI functions, and maintains the database. By creating a mapping between the deployment architecture and the functional components, it is possible to identify responsibilities for the various components and prevent inefficient operation of the system due to incompatible components.

5.13 Operational View

The operational perspective outlines how the system operates through actual situations. The chatbot remains live 24/7 and receives input from the user, then processes it via AI-based models, and provides output in real-time. The system provides seamless communication between front-end and back-end components. It provides support for a high number of concurrent user requests while providing consistent performance to them. The operational efficiency of the system is accomplished through the use of optimized-processing, low-weight models, and therefore has very low-latency in generating output back to the user.

5.14 Other Design Aspects

Several design characteristics are built into the system to have an efficient and dependable operation. The process specification defines how user input is processed and how a response is generated. The information model specification ensures that all data including symptoms and diseases are presented in a structured way. The service specification defines the API endpoints that allow communication to occur between the front end and back end of the system. In addition, there are error management mechanisms that allow for the handling of invalid user input and for managing system failure. These design characteristics of the system provide for improved maintainability, scalability, and overall performance of the system.

Chapter 6

HARDWARE, SOFTWARE AND SIMULATION

6.1 Hardware

The proposed system is a software-based application and does not involve any dedicated hardware components such as sensors, microcontrollers, or embedded circuits. Unlike traditional IoT-based systems, the chatbot operates entirely within a software environment, eliminating the need for physical interfacing.

The system is developed and executed on a general-purpose computing device such as a personal computer or laptop. All operations, including data processing, model execution, and user interaction, are handled through software modules. The integration of system components is achieved through software architecture rather than hardware connections.

Therefore, the only physical requirement for this project is a computing device capable of running the application efficiently.

6.2 Software Development Tools

The development of the AI-driven healthcare chatbot requires a combination of programming tools, frameworks, and libraries to ensure efficient implementation and performance.

An Integrated Development Environment (IDE), such as Visual Studio Code, is used for writing, debugging, and managing the application code. It provides features like syntax highlighting, debugging support, and extensions for Python and JavaScript development.

Version control is managed using Git, which helps track changes in the codebase and supports collaborative development. Flask, a lightweight Python web framework, is used to handle HTTP requests and connect the frontend with backend AI models.

The frontend of the application is developed using HTML, CSS, and JavaScript, enabling interactive user interfaces and dynamic communication with the backend through API calls.

For data processing and numerical operations, Python libraries such as Pandas and NumPy are used. The Sentence-Transformers library is utilized for generating semantic embeddings of user input, while the Transformers library is used to integrate the GPT model for generating fallback responses.

The Web Speech API is used to enable voice-based input, enhancing user interaction. Postman is used as an API testing tool to verify backend functionality and ensure proper communication between system components.

These tools are configured and integrated to support the development, testing, and deployment of the chatbot system.

6.3 Software Code and Implementation

The implementation of the healthcare chatbot system is carried out using a modular approach, where each component is designed, developed, and integrated systematically.

The backend of the system is developed using Python and the Flask framework. The application begins by importing essential libraries such as Flask, Pandas, NumPy, sentence-transformers, transformers, and langdetect, which support various functionalities including data processing, language detection, and AI model execution.

During implementation, the dataset containing diseases and associated symptoms is loaded using the Pandas library. The SentenceTransformer model is initialized to convert both dataset entries and user inputs into vector embeddings.

When a user provides input, the system first processes the text and detects the language using the langdetect module. The input is then transformed into embeddings using the sentence-transformer model. Cosine similarity is calculated between the user input embeddings and dataset embeddings using NumPy.

If the similarity score exceeds a predefined threshold, the system identifies the closest matching disease and retrieves the corresponding preventive measures. If no match is found, the system activates the fallback mechanism, where the GPT model from the transformers library generates an appropriate response.

The Flask application defines REST API endpoints, such as /chat, which receive user input in JSON format and return the chatbot response. The frontend interacts with these endpoints using JavaScript, enabling real-time communication and dynamic updates of responses on the user interface.

The implementation also includes additional features such as rate limiting, input validation, and multilingual support to enhance system reliability and security.

Each module is tested individually during development, followed by integration testing to ensure seamless interaction between frontend, backend, and AI components. This structured implementation approach ensures scalability, maintainability, and efficient performance of the system.

6.4 Simulation

The system is tested in a simulated environment without the use of physical hardware. A local development setup is used, where both frontend and backend components are executed and integrated to simulate real-world operation.

Various test scenarios are considered to evaluate system performance, including valid inputs, invalid inputs, multilingual queries, and edge cases. The system's ability to process user input, detect language, perform semantic similarity matching, and generate responses is thoroughly validated.

API testing tools such as Postman are used to simulate client-server interactions and verify the correctness of responses returned by the backend. This ensures that API endpoints function correctly under different conditions.

The simulation also involves testing real-time chatbot interaction through the user interface, ensuring smooth communication between frontend and backend components. Performance factors such as response time, accuracy, and system stability are evaluated during this phase.

This simulation approach helps in identifying potential issues, validating system functionality, and ensuring that the chatbot performs reliably before deployment.

Chapter 7

EVALUATION AND RESULTS

7.1 Test Points

The identification of test points within the various functional modules in a system may be used to confirm an implementation's correctness. Because this is software, the project specifies test points throughout the different stages of the input processing, output, and receipt of a response. The test points for the various types of functionality within a functioning application include: (1) input verification; (2) language detection; (3) generation of embedding; (4) comparison of embeddings; and (5) generation of a result based on an embedding compared to a reference or response embedding. Each of the mentioned test points will ensure that individual sections of the application will function appropriately and allow for the identification of defects if they occur.

7.2 Test Plan

For the purpose of testing the performance and function of the chatbot system, a Test Plan has been created to evaluate these factors across multiple conditions, through the use of black box and functional testing of the system.

The test plan includes the following test cases:

- TP1: The system shall accept input of valid symptoms, and reject input that is empty or invalid.
- TP2: The system shall be able to detect the input language for several languages including English, Hindi, Kannada, and Telugu.
- TP3: The system shall generate embeddings for user input, and generate similarity scores based upon a comparison of the embedding generated from the user input to the embedding for the disease as stored in the dataset.
- TP4: The system shall return the correct disease when the similarity score generated through TP3 exceeds the established threshold.
- TP5: The system shall generate a fallback response using GPT when no diseases are found that have a similarity score above the threshold.
- TP6: The system shall display the response to the user through the UI correctly.

- TP7: The system shall operate correctly without crashing or exhibiting performance degradation when multiple inputs are received at once.

The test plan verifies that all modules are tested and validated both individually and as a complete integrated system, which will confirm the total and complete functionality of each module.

7.3 Test Results

Table 7.1 Test Results of Chatbot System

Test Case	Input	Expected Output	Actual Output	Status
TP1	Empty input	Error message	Error message shown	Pass
TP2	"Fever and cough"	Disease predicted	Fever detected	Pass
TP3	Hindi input	Correct language detection	Hindi detected	Pass
TP4	Known symptoms	Correct match	Correct disease shown	Pass
TP5	Unknown input	GPT response	Generated response	Pass
TP6	Multiple inputs	System stable	No crash	Pass

The performance of the chatbot system was evaluated using multiple test cases to verify its functionality, accuracy, and robustness. The results of these test cases are presented in Table 7.1.

As shown in Table 7.1, the system successfully handles a variety of input scenarios and produces the expected outputs. The chatbot correctly displays an error message for empty inputs, ensuring proper input validation. For known symptom inputs such as "fever and cough," the system accurately predicts the corresponding disease based on semantic similarity matching.

The system also demonstrates effective multilingual capabilities, as it correctly detects and processes inputs in languages such as Hindi, along with English, Kannada, and Telugu. This enhances accessibility and usability for a wider range of users.

For inputs that do not match any known symptoms in the dataset, the system activates a fallback mechanism using a generative AI model. This allows the chatbot to produce meaningful and context-aware responses even when an exact match is not available.

7.4.Insights

The evaluation of the entire system has revealed some key insights into its performance and limitations. For example: the use of semantic similarity based on sentence-transformers has improved the prediction accuracy of diseases over what traditional keyword list approaches provide. Furthermore, the system demonstrated a high degree of system performance with fast response times with a high accuracy level when predicting disease symptoms that are known. However, the results for unknown, ambiguous symptom data were generated by the GPT-based fallback with general responses, and therefore, they may not always yield an optimal outcome. The latency and response times appeared to be minimal, as exhibited by the rapid response times for multiple inputs, thus premium for users' experiences. The system was also resilient to concurrent use by multiple users and retained consistent performance levels. The extent to which the system will perform productively, is heavily reliant upon the quality and completeness of the existing dataset. Thus, improvements to the underlying databases can positively affect the continuing productivity of the system. Furthermore, there is also an accessibility benefit through support for multiple languages; however, the consistency of generated responses may suffer slightly as a result. In summary, the system is achieving its stated goal of creating a reliable, user-friendly healthcare chatbot for the public. Future development efforts will include improved accuracy in prediction models, expansion of data sources, and the incorporation of real-time health data from providers, as contributing to the improvement of overall health care within the context of provider-patient relationships.

Chapter 8

SOCIAL, LEGAL, ETHICAL, SUSTAINABILITY AND SAFETY ASPECTS

8.1 Social Aspects

The AI health care chatbot has a large social impact because it expands access to fundamental health information. It allows users to receive general medical advice before meeting a doctor, which helps especially if they live in rural or low-income areas.

The chatbot helps to improve communication between users and medical information by providing immediate feedback on symptoms. The chatbot promotes public health by providing awareness on common diseases and their prevention.

There are some downsides to the use of chatbots, including the possibility of people relying too heavily on technology and thereby having less face-to-face contact with doctors, which could affect the quality of their medical decisions. Also, users who do not have access to a computer or the internet would not be able to use this technology, which might create a barrier between people who have access and those who do not.

In conclusion, the AI chatbot has a net positive impact on society because it improves access to healthcare, but one should not rely solely on an AI-based health care chatbot to make healthcare decisions without consulting a medical professional.

8.2 Legal Aspects

A chatbot's legal aspects are largely concerned with privacy, user consent, and compliance with regulations regarding healthcare information. Because the chatbot collects user inputs relating to a user's health status, it is essential to protect user health data integrity and privacy.

The system was designed with privacy in mind by minimizing personal health data storage, and therefore minimizes the potential risk of user privacy. The general principles followed include both data minimization and secure data processing. In the context of Indian regulations, compliance with the Digital Personal Data Protection Act (DPDPA) is critical to ensure legal handling of user data.

Liability is another critical legal risk. The chatbot will only provide users with preliminary recommendations and will not serve as a substitute for professional medical advice. Therefore, disclaimers should be present to indicate that the system is for information purposes only.

By complying with applicable legal standards, chatbot providers can establish their users' trust while eliminating potential misuse of the system.

8.3 Ethical Aspects

In developing and implementing AI-driven healthcare systems, ethical considerations are an important aspect of creating these solutions. The project has a primary ethical duty to provide accurate, unbiased information that will not be harmful to users.

The ethical challenges presented by this kind of project primarily stem from algorithm bias, which may arise due to an incomplete or unevenly distributed dataset used in training. In such cases, the model may yield false or misleading results. Using reliable and diverse datasets, therefore, is vital.

In addition, transparency is another important ethical consideration. Users must be clearly informed that the system is an AI chatbot and not a certified doctor/midwife. This provides a way for users to make educated choices when using it.

There should also be fairness through equal access to the system regardless of the user's language or background. Providing support for multiple languages increases access to all users and provides inclusivity.

To sum up, ethical design will ensure that the system puts the needs of its users first, and builds trust among its users.

8.4 Sustainability Aspects

Focusing on resource efficiency and minimization of negative environmental impact, the sustainable aspect of the project emphasizes efficient resource use and minimizing environmental impact.

As the system relies on software-based solutions, it does not consume any tangible materials and hence, minimizes resource consumption.

Use of lightweight AI Models (Distil GPT2) helps mitigate computational requirements, which reduces the energy consumed.

With optimally coded algorithms and efficient coding practices, additional resource efficiency is achieved as well.

The system has been developed as scalable and maintainable for long-term use. Energy usage can be efficiently deployed through cloud or local server infrastructure.

Additionally, the system helps to reduce the number of unnecessary appointments to hospitals due to minor concerns and in turn has helped reduce the amount of transportation-related greenhouse gas emissions.

The projects' focus on efficient usage of resources through sustainable practices is aligned with and promotes sustainable development principles.

8.5 Safety Aspects

It is extremely important for the safety of users of the health care chatbot platform as giving incorrect or inaccurate information could potentially harm their health; therefore, various safety measures are implemented on the platform to lessen the occurrence of issues and to guarantee that the platform is functioning properly as intended.

The platform contains built-in mechanisms to ensure the chatbot has accurate and reliable contents; for instance, the platform uses input validation to prevent the chatbot from accepting user-created data (such as harmful code, invalid data) that would negatively affect the health care functionality of the chatbot. The platform also incorporates many different forms of protection against the most common types of attacks (e.g., rate limiters, input sanitization) that would negatively impact the way the platform functions.

The platform indicates limitations on the health care information it provides as it does not provide or create information about a user's life-saving medical procedures. The platform will provide general health care advice, but will refer users to the proper medical authorities for serious medical concerns.

From a cybersecurity perspective, the platform does not retain users' sensitive data; hence, the likelihood of user data breach is diminished. This platform continually tests its capabilities for the delivery of service in numerous scenarios.

Overall, the safety and security measures of the system provides users with access to accurate, legitimate health care information while minimizing the potential impact on their health, while using the chatbot

CONCLUSION

An AI-driven public health chatbot provides a comprehensive, scalable method of delivering basic awareness of healthcare through advanced artificial intelligence techniques.It uses a hybrid architecture consisting of a semantic similarity based disease prediction mechanism utilizing SentenceTransformers and a generative fallback mechanism that utilizes DistilGPT2. This architecture provides the system with the ability to accommodate both structured queries (based on symptoms alone) and unstructured queries (ambiguous or incomplete).

The chatbot meets the objectives for the project as identified in the Introduction. The chatbot accepts user symptoms; uses AI models to process user symptoms, predict potential diseases, and recommend methods for preventing those diseases. Additionally, when combined with additional means of human computer interaction such as multilingual support and voice input, users from various backgrounds and languages can now access and use the system.

The evaluation results of the chatbot show that the use of semantic similarity methods provides accurate results when input from the user contains known symptoms. However, the generative fallback mechanism provides usable or helpful answers to users who do not identify known symptoms. The system exhibits low latency and stable performance and therefore confirms its viability for real time use. The system also demonstrates a level of robustness with respect to the various ways of inputting information into the chatbot and the numerous interactions by multiple users. Enhancements in future releases will include broadening the dataset to encompass more types of illnesses, incorporating medical-specific domain languages into their models (e.g., using a system to understand how to best use medical terms), and increasing understanding of context and communication by implementing memory functions so users can have conversations with the bot like a friend. Future usability and effectiveness could be further enhanced by implementing a mobile application version of the bot and providing access to real-time healthcare services through the use of devices such as smartphones or tablets.

In summary, the system built will show how AI technology can be effectively blended with user functionality to develop a bot that is efficient, scalable, and user-friendly. The bot would be a valuable resource for increasing public health awareness in areas where there is limited access to healthcare, and it has excellent potential for future enhancement and real-world deployment.

REFERENCES

[1] G. Muniselvam and S. Anushalakshmi, "AI-Powered Virtual Job Interview Simulation System Using NLP," International Research Journal of Modernization in Engineering, Technology and Science, vol. 7, no. 6, pp. 614–616, Jun. 2025.

[2] V. Karthick, S. Raghav, and R. Anitha, "Development of a Web-Based Selector Applicant Simulation Software with Integrated Proctoring System," International Journal for Research in Applied Science and Engineering Technology (IJRASET), vol. 13, no. 4, Apr. 2025.

[3] J. R. Alapati et al., "Web-Based Selector – Applicant Simulation System," International Journal of All Research Education and Scientific Methods (IJARESM), vol. 13, no. 3, Mar. 2025.

[4] G. Borade et al., "Smart Proctor Hub – Online Exam Proctoring System," International Research Journal of Modernization in Engineering, Technology and Science, vol. 7, no. 4, Apr. 2025.

[5] R. Dhadwe et al., "Web-Based Selector – Applicant Simulation Software," International Journal of Emerging Technologies and Innovative Research, 2025.

[6] J. Devlin, M. W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," Proceedings of NAACL-HLT, 2019.

[7] T. Wolf et al., "Transformers: State-of-the-art Natural Language Processing," Proceedings of EMNLP, 2020.

[8] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," Proceedings of EMNLP, 2019.

[9] A. Vaswani et al., "Attention Is All You Need," Advances in Neural Information Processing Systems (NeurIPS), 2017.

[10] A. Radford et al., "Language Models are Unsupervised Multitask Learners," OpenAI, 2019.

[11] K. Huang et al., "ClinicalBERT: Modeling Clinical Notes and Predicting Hospital Readmission," arXiv preprint arXiv:1904.05342, 2020.

[12] P. Rajpurkar et al., "CheXNet: Radiologist-Level Pneumonia Detection on Chest X-Rays with Deep Learning," arXiv preprint arXiv:1711.05225, 2017.

[13] E. M. Bender et al., "On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?" Proceedings of FAccT, 2021.

[14] E. Topol, “High-performance medicine: the convergence of human and artificial intelligence,” Nature Medicine, vol. 25, no. 1, pp. 44–56, 2019.

[15] K. Denecke et al., “Clinical Text Mining using Natural Language Processing,” Journal of Biomedical Informatics, 2018.

[16] A. S. Miner et al., “Conversational agents and healthcare: A systematic review,” Journal of Medical Internet Research, vol. 18, no. 7, 2016.

[17] A. Esteva et al., “A guide to deep learning in healthcare,” Nature Medicine, vol. 25, no. 1, pp. 24–29, 2019.

[18] World Health Organization, “Ethics and Governance of Artificial Intelligence for Health,” WHO, 2021.

19] Ministry of Electronics and Information Technology, “Digital Personal Data Protection Act,” Government of India, 2023.

[20] United Nations, “Sustainable Development Goals,” UN Department of Economic and Social Affairs, 2015.

BASE PAPER

Adamopoulou, E. and Moussiades, L., 2020. An overview of chatbot technology. IFIP International Conference on Artificial Intelligence Applications and Innovations, pp.373–383.
This paper provides a comprehensive overview of chatbot systems, including rule-based, retrieval-based, and generative chatbot architectures. It discusses how natural language processing techniques are used to understand user input and generate responses. The paper also highlights the limitations of traditional chatbot systems and the need for advanced AI-based approaches.
The concepts presented in this paper are closely related to the proposed project, as it utilizes NLP techniques and AI models to develop an intelligent healthcare chatbot. The hybrid approach used in the project, combining semantic similarity and generative models, aligns with the advancements discussed in this paper.

APPENDIX

A. Project Report – Similarity Report

turnitin Page 2 of 10 - Integrity Overview Submission ID trn:oid:::1:3547449126

1% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Match Groups

- 2 Not Cited or Quoted 1%
  Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations 0%
  Matches that are still very similar to source material
- 0 Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

1% Internet sources
0% Publications
0% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

B. Repository Link - https://github.com/sudeepkasetty/CSE-159-AI-driven-public-health-chatbot-for-disease-awareness.git

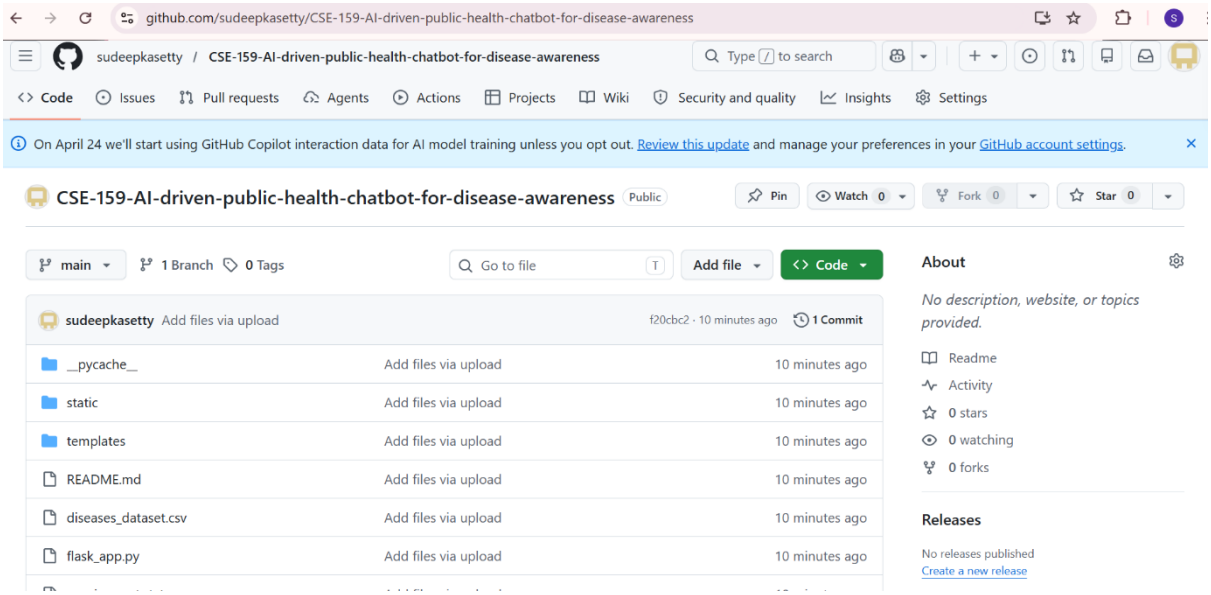

Fig B.1 GitHub Repository

C. Project Output

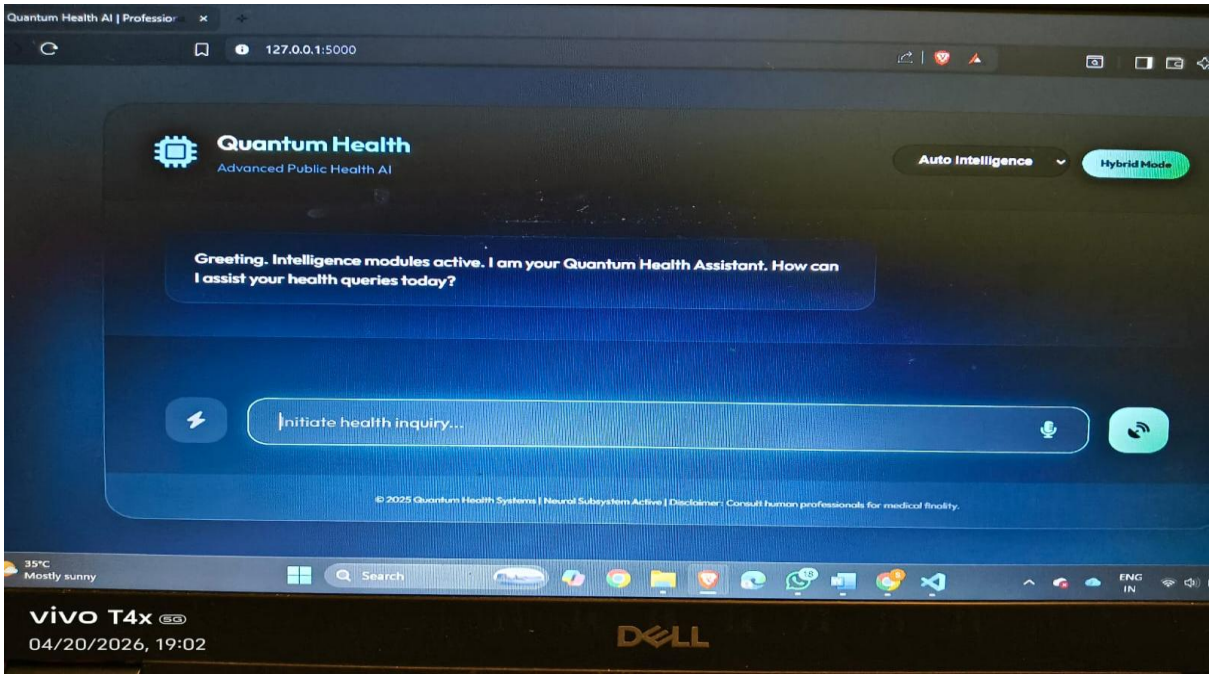

Fig C.1 Home Page

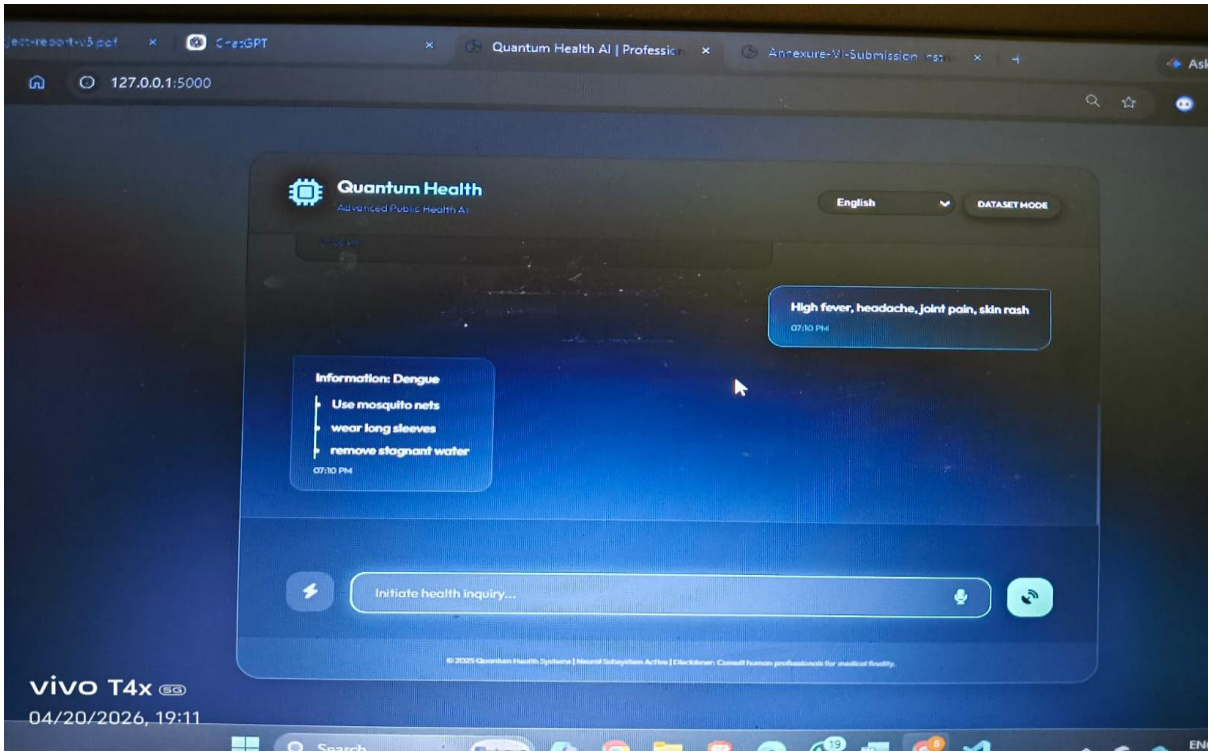

Fig C.2 Multilingual Interface (English)

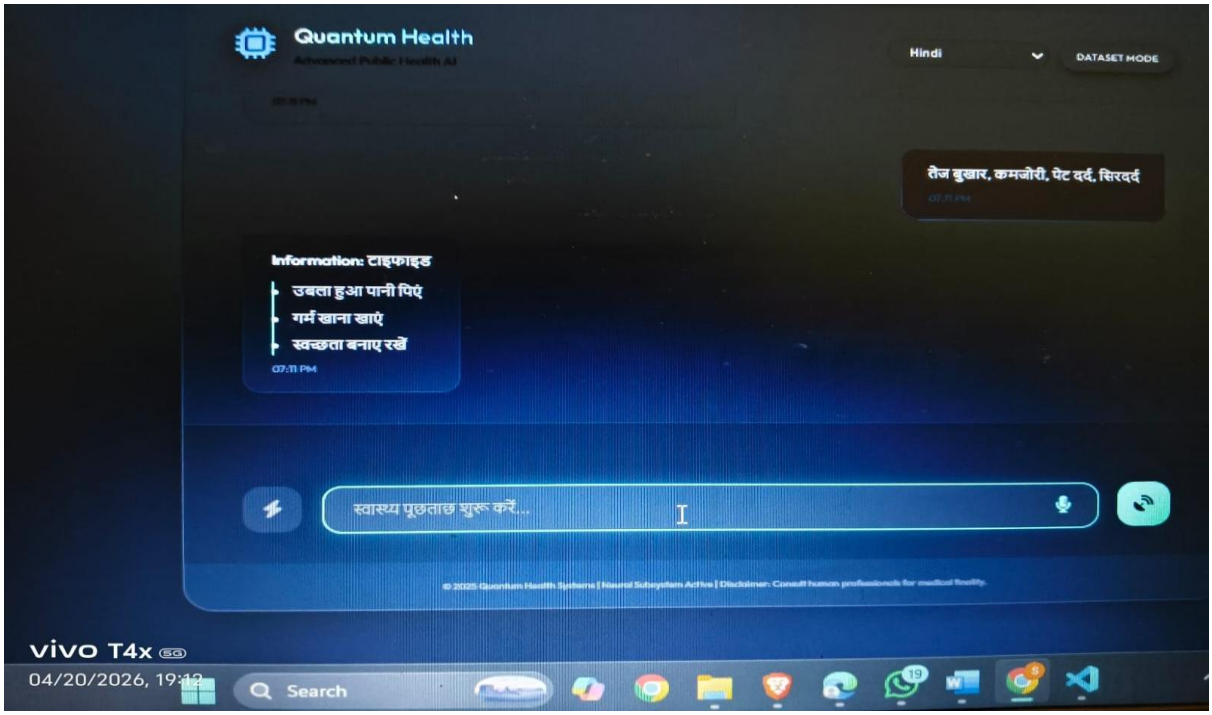

Fig C.3 Multilingual Interface (Hindi)

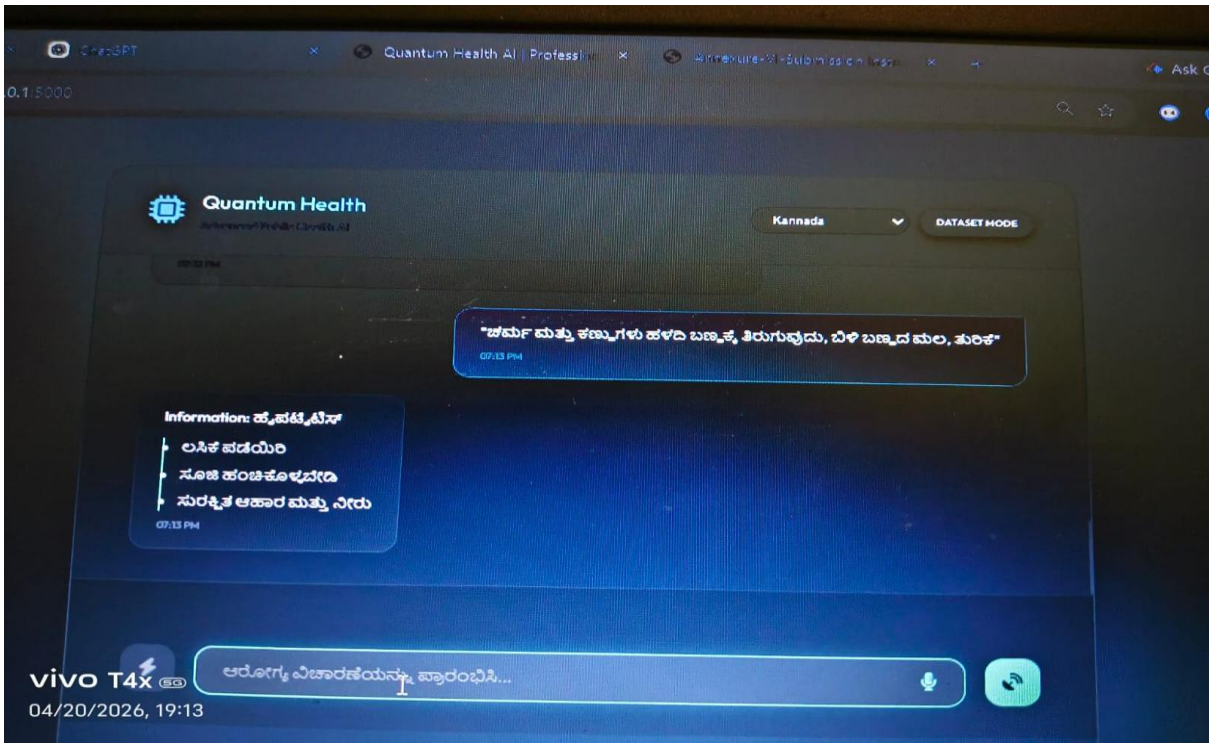

Fig C.4 Multilingual Interface (Kannada)

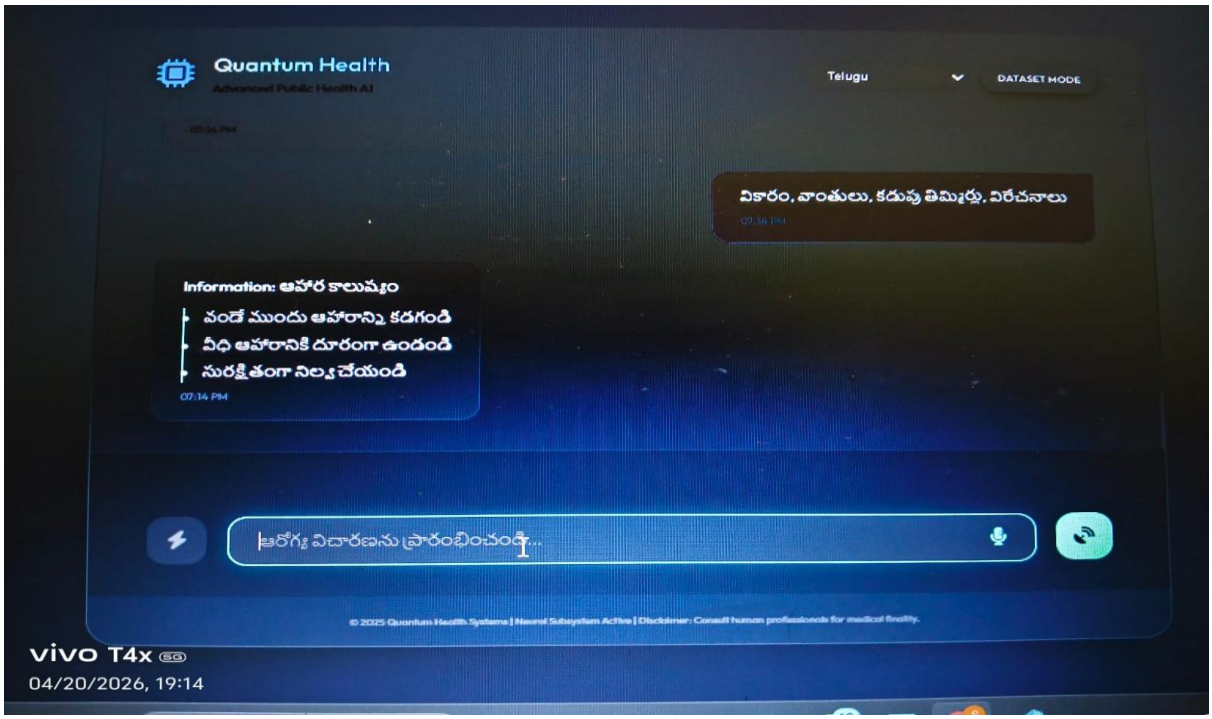

Fig C.5 Multilingual Interface (Telugu)

.